

PilotFish FTP / SFTP Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **remote outbox**. When a transaction finishes local processing, this transport uploads the message body as a file on an FTP, FTPS, or SFTP server — choosing the path, conflict behavior, and protocol stack.

FTP / SFTP Transport

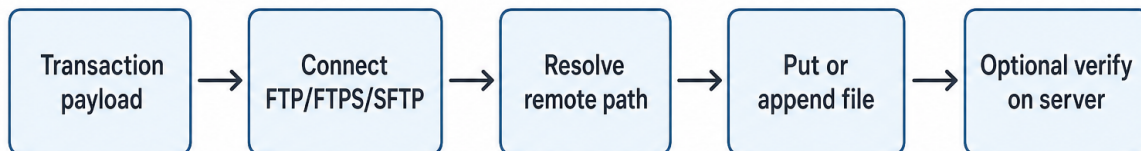


Figure 1: At a glance (plain English)

- **Uploads** transaction bytes to a remote directory or full path.
- **Supports** regular FTP, explicit/implicit FTPS, and two SFTP stacks (SSHTools / JSCH).
- **Handles** name conflicts via unique suffix unless overwrite/append is set.
- **Optionally** keeps connections, retries, verifies remote presence, and times out the whole put.

Purpose

This document is a **deep dive** into the **FTP / SFTP** transport as shipped in **eip.war.hs.26R1.11**: protocol selection, path resolution, put/append, keep-alive handlers, timeouts, and FTPS certificate options.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-ftp-1.1.2.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.file.ftp.FTPTransport</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../FTPTransport.class</code>
Transport type string	FTP / SFTP
Description	Stores data as files in a specified directory on a remote FTP server.
Help ID	<code>console.transport.ftp</code>
Categories	FILESYSTEM, FTP, WEB, SPECIAL_PROTOCOL

Related classes:

Class	Role
<code>FTPTransport</code>	Timeout FutureTask wrapper, keep-connection handler map
<code>FtpTransportHandler</code> / factory	Protocol-specific handlers
<code>FtpFtpsTransportHandler</code>	Commons-Net FTP/FTPS store/append
<code>SftpTransportHandler</code>	SSHTools SFTP put + optional Set File Timestamp
<code>SFTPJschTransportHandler</code>	JSCH SFTP put + Write From Home
<code>FtpTransportCommons</code>	Path build, unique-name collision logic
<code>FtpCommons</code>	Credentials, FTPS keystore/truststore, progress monitors

Derived from WAR bytecode (CFR). Narrative follows `executeTransport` / handler puts.

1. Conceptual model

`FTPTransport.executeTransport` (synchronized on module)

|

v

`getTransportHandler(keepConnection?)`

|

```

    v
FutureTask on "FTP Transport Runner Thread"
    | optional Timeout seconds
    v
handler.connect if needed
handler.executeTransport
    |
    +-- resolve remote path (full path OR dir+name+ext)
    +-- store / append / SFTP put modes
    +-- optional Check for File
    |
    finally: close unless Keep Connection

```

UI descriptor title remains legacy "FTPTransportOld".

2. High-level behavior

1. Read **Connection Timeout** (Timeout, seconds). 0 disables FutureTask time cap.
2. Resolve handler for **FTP Type** (new each call, or reuse from liveHandlers when **Keep Connection**).
3. Synchronize on transport instance; start NTM thread running `execute(handler, data)`.
4. `ftpFuture.get(timeout)` or unbounded `get()`.
5. Interrupted/Timeout -> cancel future, force `keepConnection=false`, throw `EIPEException("FTP connection timeout / deadlock encountered")`.
6. `ExecutionException` -> `EIPEException("Unable to execute FTP transport")`.
7. `finally`: close handler unless keep-connection still true.
8. Inner `execute`: synchronized on handler; connect if needed; interrupt checks; `handler.executeTransport(data)`.
9. `systemShutdown`: close all live connected handlers.

Path resolution (`FtpTransportCommons.getNameFromTargetPath`)

1. **Specify full file path = Enabled** -> use `FullPathToFile` as remote target string.
2. Else build `FileName` + optional `.` + `FileExtension`.
3. If **neither** Overwrite nor Append:
 - List matching remotes via handler `getMatchingFiles`.
 - If matches exist, pick unique name base-N.ext via `appendIndexToFilename`.
4. Else / if no unique candidate: `ensureEndingSeparator(TargetDirectory) + fileName` (`cwd/.///.` treated as no directory prefix when generating uniqueness list path).

Append on SFTP stacks often uses `generateFullNameFromConfig` (no uniqueness rename) then resume/append-style put flags.

3. Configuration reference

3.1 Basic path & protocol

UI label	Tag	Type	Default	Tab	Required	Nuances
FTP Type	FTPTransportType	ChoiceType	Regular FTP	Basic	Yes	Five stacks (below)
Host	Host	String	""	Basic	Yes	Required
Port	Port	Integer	21	Basic	Yes	21
Target directory	TargetDirectory	String	""	Basic	Yes*	FTP/FTPS; 22 typical SFTP Some servers require trailing /; . / ./ = cwd; / = root
Target file name	FileName	String	""	Basic	Yes*	Base name
Target file extension	FileExtension	String	""	Basic	No	Dot prepended when non-empty
Specify full file path	UseFullFilePath	Boolean	Disabled	Basic	No	Enabled / Disabled
Path to file	FullPathToFile	String	""	Basic	Yes*	When full path enabled

FTP Type choices: Regular FTP; FTP over SSL (Explicit); FTP over SSL (Implicit); Encrypted FTP (SSH/SFTP); Encrypted FTP (JSCH SSH/SFTP).

Dependency panels

- UseFullFilePath=Disabled -> directory, name, extension.
- UseFullFilePath=Enabled -> full path only.

3.2 Advanced

UI label	Tag	Type	Default	Tab	Nuances
Connection Timeout	Timeout	Time (sec)	300	Advanced	Entire FutureTask; 0 disables
Socket Timeout	Sftp_SocketTimeout	Time (sec)	0	Advanced	Connect wait; SFTP <=0 -> library default ~30s
Retry Connection	RetryConnection	Boolean	false	Advanced	Second connect attempt after delay

UI label	Tag	Type	Default	Tab	Nuances
Retry Delay	<code>RetryDelayTime</code>	<code>int</code>	<code>1</code>	Advanced	Visible when <code>retry</code> true; <code>-1</code> = no delay
Keep Connection	<code>KeepConnection</code>	<code>Boolean</code>	<code>false</code>	Advanced	Handler cached by factory key; closed on shutdown/timeout
Passive Mode	<code>UsePassiveMode</code>	<code>Boolean</code>	<code>true</code>	Advanced	FTP/FTPS types only
SSL Protocol	<code>SSLProtocolChoice</code>	<code>Choice</code>	<code>TLS</code>	Advanced	FTPS; choices SSLv2/SSLv3/TLS
Data Channel Protection Level	<code>ProtectionChoice</code>	<code>Choice</code>	<code>Private (P)</code>	Advanced	FTPS PROT: P/C/S/E/Not Specified
File Type	<code>FileType</code>	<code>Choice</code>	<code>Binary</code>	Advanced	Binary vs Text (ASCII) on FTP/FTPS
Append to File	<code>AppendToFile</code>	<code>Boolean</code>	<code>false</code>	Advanced	Append/resume style put
Overwrite existing	<code>Overwrite</code>	<code>Boolean</code>	<code>false</code>	Advanced	Shown when <code>Append=false</code> ; skips unique rename when true
Check for File	<code>CheckForFileAfterWriting</code>	<code>Boolean</code>	<code>false</code>	Advanced	Post-put remote list validation
Fail on FALSE return result	<code>FAIL_ON_FALSE_RESULT</code>	<code>Boolean</code>	<code>true</code>	Advanced	FTP/FTPS: treat <code>storeFile/appendFile</code> false as error
JSCH Write From Home	<code>WriteFromHome</code>	<code>Boolean</code>	<code>false</code>	Advanced	JSCH only: <code>cd</code> home before put
Set File Timestamp	<code>SetFileTime</code>	<code>Boolean</code>	<code>true</code>	Advanced	SSHTools SFTP: set <code>atime/mtime</code> after put

Note: constant `EnsureSlash` exists on `FTPTransport` but is **not** registered in the configuration descriptor (unused UI option in this WAR).

3.3 Credentials (`FtpCommons.getCredentialsDescriptor`)

UI label	Tag	Type	Default	Tab	Nuances
User name	<code>UserName</code>	<code>String</code>	<code>" "</code>	Credentials	Required

UI label	Tag	Type	Default	Tab	Nuances
Password	Password	Password	""	Credentials	Optional with key auth
Use key file	KeyAuthentications	Boolean	false	Credentials	SFTP types only (dependency)
Select key file	KeyFilePath	Path	""	Credentials	When key auth; falls back to <code>sftp_key.dat</code>

Key auth + path items depend on FTP Type = SSH/SFTP or JSCH SSH/SFTP.

3.4 Certificates (FTPS)

UI label	Tag	Type	Default	Tab	Nuances
Use Key Store	UseKeyStore	Boolean	false	Certificates	Client certs (2-way SSL)
Key Store	KeyStoreFilePath	Path	""	Certificates	Required when use key store
Key Store Type	KeyStoreFileType	String	""	Certificates	Recommend JKS
Key Store Password	KeyStoreFilePassword	Password	""	Certificates	
Use Trust Store	UseTrustStore	Boolean	false	Certificates	Validate server cert
Trust Store	TrustStoreFilePath	Path	""	Certificates	
Trust Store Type	TrustStoreFileType	String	""	Certificates	
Trust Store Password	TrustStoreFilePassword	Password	""	Certificates	

`readConfiguration` auto-sets `UseKeyStore/UseTrustStore` to true when legacy path/type values exist without the boolean flags.

3.5 Inherited AbstractTransport

Behavior	Detail
Lifecycle	<code>sendData -> checkRequiredData -> executeTransport -> complete/error</code>
Batch sync	<code>false</code>
Parent routing	Available via <code>AbstractTransport</code> parent accessors

4. Protocol-specific put algorithms

FTP / FTPS (`FtpFtpsTransportHandler`)

- Non-append: `storeFile(target, stream)`.
- Append: `appendFile(target, stream)`.
- If `!outputStreamed && FAIL_ON_FALSE_RESULT` -> `EIException` with reply code.
- Requires positive intermediate/completion reply handling (`completePendingCommand` when intermediate).
- Optional `CheckForFileAfterWriting` lists remote; missing file -> `IOException`; ambiguous names -> warn logs.

SFTP SSHTools (`SftpTransportHandler`)

- Non-append: `sftpClient.put(is, targetName)`.
- Append: `generateFullNameFromConfig`; if remote exists put with resume flag `1L`, else normal put.
- Optional `SetFileTime` updates last access/modified via subsystem attributes (errors logged, not always fatal).
- Check-for-file throws `EIException` on name mismatch (stricter than FTP warn path).
- Certain `SSHServerAPI.SFTP.fxp_attrs` errors logged as warn and swallowed (legacy quirk).

SFTP JSCH (`SFTPJschTransportHandler`)

- Optional `WriteFromHome` -> `sftpChannel.cd(home)` before put.
- Non-append put with progress monitor; append uses put mode 2.
- Check-for-file via `ls` + `SFTPJschUtils` entry helpers.

5. Transaction attributes

5.1 Written by this transport

None specifically in the handler put path (unlike Directory File's `TargetFile*` attributes).

5.2 Consumed

Configuration values only (plus any OGNL/enhanced substitutions the manager applies). Credentials/key material resolved through `ResourceLoader`.

6. Worked example — SFTP unique name drop

- FTP Type: Encrypted FTP (SSH/SFTP)
- Host `files.partner.com`, Port 22
- Target directory `/inbound/claims/`
- File name `claim`, extension `xml`
- Append false, Overwrite false

- Keep Connection false, Timeout 300
- Check for File true

Behavior:

1. Handler connects SFTP.
2. If `claim.xml` exists remotely, uploads as `claim-1.xml` (or next free index).
3. Lists remote to verify presence; mismatch fails the transaction.
4. Connection closed after put.

7. Known quirks and caveats

1. **Instance + handler synchronization** — concurrent puts on one module instance serialize; keep-connection keys share handlers carefully.
2. **Timeout cancels keep-alive** for that failure path (`keepConnection=false` in catch).
3. **Unique rename only when both Append and Overwrite are false.**
4. **SFTP append path skips uniqueness** (`generateFullNameFromConfig`).
5. **FAIL_ON_FALSE_RESULT** — some FTP servers return false despite success; uncheck to ignore.
6. **Trailing slash** — still server-dependent despite `ensureEndingSeparator` helpers.
7. **EnsureSlash tag unused** in UI for this WAR.
8. **Descriptor title FTPTransportOld** is cosmetic leftover.
9. **Set File Timestamp** meaningfully applied on SSHTools SFTP handler; not the same on JSCH put path in decompile.
10. **Stream consumed** by remote put; no local `TargetFile` attribute published.

8. Operational recommendations

Goal	Guidance
Prefer modern SFTP	Use SSHTools or JSCH with key auth; disable unused FTPS cert tabs
Flaky networks	Enable Retry Connection; tune Socket Timeout; keep Connection Timeout above worst put
Exactly-once names	Overwrite false + Append false for numeric suffix uniqueness
Partner requires home-relative JSCH	Enable JSCH Write From Home
False FTP completions	Start with Fail on FALSE = true; disable only after confirming server quirk
High volume	Keep Connection true for same host/user; monitor stale sessions; rely on <code>systemShutdown</code> cleanup

9. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
FTP / SFTP transport Path / uniqueness FTP/FTPS handler SFTP handler JSCH handler Credentials / certs Abstract transport Module JAR	 com/pilotfish/eip/modules/file/ftp/FTPTransport.java com/pilotfish/eip/modules/file/ftp/FtpTransportCommons.java com/pilotfish/eip/modules/file/ftp/FtpFtpsTransportHandler.java com/pilotfish/eip/modules/file/ftp/SftpTransportHandler.java com/pilotfish/eip/modules/file/ftp/SFTPJschTransportHandler.java com/pilotfish/eip/modules/file/ftp/FtpCommons.java com/pilotfish/eip/extend/AbstractTransport.java war-pipeline/jars/eip-26R1.11/xcs-modules-ftp-1.1.2.jar

10. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11

-> xcs-modules-ftp-1.1.2.jar

-> CFR decompile

-> Documents/Transports/26R1.11/PilotFish-FTP-SFTP-Transport-Reference-26R1.11.(md|pdf)

Deep-dive documentation from WAR decompile – FTP / SFTP Transport – 26R1.11 – August 2026.